

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

QUESTIONS: 2 Duration: 45 Minutes Total Marks: 20

COURSE OUTCOME COVERED

CO1: *Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)*

Assessment Tool Weightage: 10%

Annexure A

Descriptive Written Test

Code of Conduct:

I S. Karthikeyan / 192419048 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

CSA17 — ARTIFICIAL INTELLIGENCE

Assessment Tool 3 — Descriptive Written Test: Answers

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively

Q1. Smart Home Automation — Types of Intelligent Agents

A smart home system manages lighting, temperature, and security based on user behaviour and environmental conditions, responding to real-time inputs (motion, temperature, time of day) while learning user preferences over time.

1. Simple Reflex Agent

Selects an action based only on the current percept, using fixed condition–action ("if–then") rules, with no memory of the past.

- In this system: "IF motion detected THEN turn ON the light"; "IF temperature > 28°C THEN turn ON the AC"; "IF a door opens while armed THEN sound the alarm."
- Limitation: it cannot use context — e.g., it may turn a light on even though it is already bright daylight, or cannot tell a genuine intrusion apart from a resident who just stepped out, because it only ever reacts to the instantaneous percept.

2. Model-Based Reflex Agent

Maintains an internal model of aspects of the world that are not visible in the current percept alone, to handle partial observability.

- In this system: keeps track of "is anyone currently home," "which rooms are occupied," and "the recent temperature trend," built up from a history of motion/door sensor readings.
- This lets it, for example, avoid triggering a false alarm right after it has internally noted the resident leaving through the front door, or begin cooling a room slightly before the threshold is crossed because its model shows temperature is trending upward.

3. Goal-Based Agent

Chooses actions by considering whether they lead toward an explicit goal, using planning/search over possible future states rather than fixed rules.

- In this system: given a goal such as "the living room should be at a comfortable temperature by 6 pm when the resident usually returns," the agent plans ahead — e.g., starting the AC at 5:15 pm — instead of waiting for the temperature threshold to be crossed.
- For security, a goal like "the house must be fully secured before occupants are asleep" causes the agent to actively check and lock doors and arm the alarm, rather than only reacting once a sensor is triggered.

4. Utility-Based Agent

When several actions could all satisfy a goal, this agent picks the one that maximises a utility function balancing multiple, sometimes competing, objectives.

- In this system: weighs comfort against energy cost against security priority at once — e.g., choosing between full-power cooling (max comfort, high cost) and a moderate eco-mode (slightly less comfort, lower cost) depending on time-of-day electricity rates, or deciding whether to keep a light on for a security appearance versus saving energy while the house is empty.
- This handles genuine trade-offs that a goal-based agent cannot represent, since goal-based reasoning only distinguishes "goal met" from "goal not met," with no notion of one solution being better than another.

Justification: A Hybrid, Learning Architecture Is Most Effective

No single agent type alone is sufficient for comfort, energy efficiency, and security together. The most effective design is a hybrid, layered architecture that uses all four types for what each does best, wrapped in a learning element that personalises the system to the residents over time:

- Simple reflex layer — handles urgent, safety-critical responses with minimal latency (e.g., an immediate smoke/fire alarm), where waiting to "think" would be dangerous.
- Model-based layer — supplies the occupancy/trend context every higher layer needs, since the raw current percept alone is not enough to make good decisions (partial observability).
- Goal-based layer — plans ahead for scheduled comfort and security goals (pre-cooling, securing the house before occupants sleep).
- Utility-based layer — arbitrates between comfort, energy efficiency, and security whenever multiple actions all satisfy the current goal but differ in desirability.
- Learning element — observes the residents' manual overrides and habits over time (e.g., they always nudge the temperature down at night) and updates the internal model and utility function accordingly, which is exactly what lets the system "learn user preferences over time" as the scenario requires.

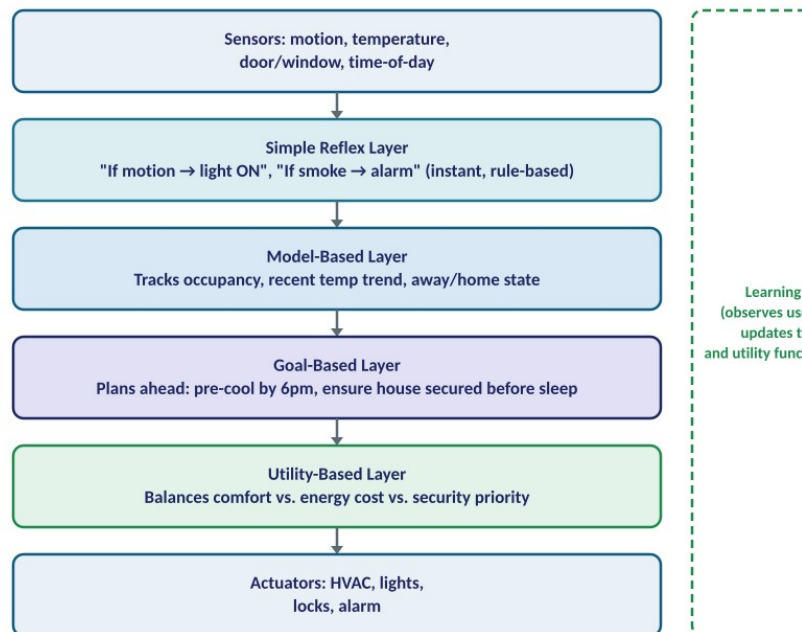


Figure 1 — Hybrid layered agent architecture for the smart home system: reflex rules handle urgent safety cases, the model tracks hidden state, goals drive planning, utility resolves trade-offs, and a learning element personalises the whole stack over time.

In short: simple-reflex behaviour alone is too rigid, and goal-based reasoning alone cannot weigh comfort against energy cost — so the hybrid, learning agent above is justified as the most effective design for this system.

Q2. Robot in an Unknown Maze — UCS, IDS, and Agent Design

A robot is placed in an unknown maze and must find a path to the exit with no prior knowledge of the layout.

How Uniform Cost Search (UCS) Helps

UCS expands the frontier node with the lowest cumulative path cost $g(n)$ first, using a priority queue. It is complete and optimal whenever all step costs are non-negative, so it is guaranteed to find the least-cost path to the exit — useful if some maze cells are more costly to cross than others (e.g., rough terrain), not just "fewest steps."

How Iterative Deepening Search (IDS) Helps

IDS repeatedly runs a depth-limited depth-first search with an increasing limit (0, 1, 2, ...) until the exit is found. This combines DFS's low memory usage with BFS's completeness and optimality (for uniform step cost), which fits a maze where every move (one cell forward) costs the same.

Advantages and Disadvantages

Algorithm	Time Complexity	Space Complexity	Advantages	Disadvantages
UCS	$O(b^{(1+\lceil C^*/\epsilon \rceil)})$	$O(b^{(1+\lceil C^*/\epsilon \rceil)})$ — exponential, must store the frontier	Optimal & complete even with varying step costs; finds true least-cost exit path	Can need a very large amount of memory in a big/unknown maze; needs edge costs to be known as each cell is reached
IDS	$O(b^d)$	$O(b \cdot d)$ — linear in solution depth	Very memory-efficient (ideal for a robot's limited onboard memory); complete & optimal for uniform step cost; needs no prior knowledge of maze depth	Re-explores shallow nodes on every iteration (time overhead); like BFS, only optimal when all step costs are equal

Relating the Problem to Types of Intelligent Agents

- A simple reflex agent (e.g., a "wall-following"/right-hand-rule robot) can escape many simple mazes with no search or memory at all, but fails on mazes with loops or islands and cannot guarantee the shortest exit path.
- A model-based reflex agent keeps an internal map of the maze cells already visited, letting the robot avoid dead ends and infinite loops — this internal map is a prerequisite for running UCS or IDS at all, since neither algorithm can search a graph the robot has no memory of.
- A goal-based agent is the natural fit for "find a path to the exit": it explicitly runs UCS or IDS over its (growing) internal maze model to reach the known goal, choosing actions based on progress toward that goal rather than fixed rules.
- A utility-based agent becomes relevant if the maze offers multiple viable exits or paths with different trade-offs (e.g., a shorter but narrower route vs. a longer but safer one), letting the robot choose the best option rather than merely a valid one.

Agent design directly affects decision-making here: without the model-based component's memory, the robot has no internal graph to search at all, so the choice of agent architecture determines which search strategies can even be used — a model-based, goal-based agent is a precondition for deploying UCS or IDS in an unknown maze.

Recommended Combination and Justification

Recommended: a model-based, goal-based agent using Iterative Deepening Search, for a maze with uniform per-move cost.

- The maze is unknown, so the model-based component is essential — it must build the map from what it discovers as it explores; this component is required regardless of which search algorithm is layered on top.
- Since the maze's size and solution depth are unknown in advance, IDS's linear $O(b \cdot d)$ memory requirement is far more practical for a robot with limited onboard memory than UCS's potentially exponential memory needs, while still guaranteeing completeness and optimality for the maze's uniform move cost.
- If the maze instead had meaningfully different traversal costs per cell (e.g., some terrain slower or more energy-costly to cross), a model-based, utility-based agent using UCS would become the better choice instead, since UCS natively optimises for weighted cost while IDS assumes every move costs the same.
- Given the question's plain framing — an unknown maze, simply looking for the exit — the default, well-justified recommendation is a model-based, goal-based agent paired with Iterative Deepening Search, with UCS as the natural upgrade path if variable move costs are introduced.